



PROGRAMACIÓN EN AMBIENTE CLIENTE SERVIDOR

Ing. Alex Ramírez Galindo

Práctica de evaluación primer parcial

Que tales jóvenes, en este documento les documentaré la práctica final del primer parcial. Cerraremos el tema de Sockets de comunicación en un ambiente web.

Trabajaremos como servidor con Node.js y el editor de Visual Code.

Iniciamos creando una carpeta en la unidad C de su computadora, llamada ChatNodeAlex (cambiarlo por su nombre) y lo abren desde Visual Code (ejecutarlo con administrador).

Ya con la carpeta abierta en VS, abrir una terminal para iniciar la configuración del proyecto. Escribir el comando npm init (enter) y colocar los siguiente parámetro (después de cada parámetro, da enter), como lo muestro en la siguiente imagen

```
PS C:\ChatNodeAlex> npm init
This utility will walk you through creating a package.json file.
It only covers the most common items, and tries to guess sensible defaults.

See `npm help init` for definitive documentation on these fields
and exactly what they do.

Use `npm install <pkg>` afterwards to install a package and
save it as a dependency in the package.json file.

Press ^C at any time to quit.
package name: (chatnodealex)
version: (1.0.0)
description: Chat en nodejs
entry point: (index.js)
test command:
git repository:
keywords:
author: Alex Ramírez Galindo
license: (ISC)
About to write to C:\ChatNodeAlex\package.json:

{
  "name": "chatnodealex",
  "version": "1.0.0",
  "description": "Chat en nodejs",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "Alex Ramírez Galindo",
  "license": "ISC"
}

Is this OK? (yes) yes
PS C:\ChatNodeAlex>
```

Dar enter.
Desde test command hasta
keywords solo dar enter en
cada parámetro
En autor, coloca tu nombre
completo





Una vez finalizado la configuración, se crear el archivo package.json dentro de la carpeta.

Luego, en la misma terminal vamos a instalar el framework de express, con el siguiente comando

```
PS C:\ChatNodeAlex> npm install express --save  
  
added 68 packages, and audited 69 packages in 4s  
  
16 packages are looking for funding  
run `npm fund` for details  
  
found 0 vulnerabilities  
PS C:\ChatNodeAlex>
```

También, instalar una librería que la usaremos para que cada vez que se cambie una línea de código, la aplicación se restaure automáticamente

```
PS C:\ChatNodeAlex> npm install nodemon --save-dev  
  
added 27 packages, and audited 96 packages in 3s  
  
20 packages are looking for funding  
run `npm fund` for details  
  
found 0 vulnerabilities  
PS C:\ChatNodeAlex>
```

Y configuramos dicha librería en el archivo package.json, para eso agregar la siguiente línea de código para indicarle que ese será el archivo que se ejecutará.

```
package.json > ...  
1 {  
2   "name": "chatnodealex",  
3   "version": "1.0.0",  
4   "description": "Chat en nodejs",  
5   "main": "index.js",  
6   "scripts": {  
7     "test": "echo \"Error: no test specified\" && exit 1",  
8     "start": "node index.js"  
9   },  
10  "author": "Alex Ramirez Galindo",  
11  "license": "ISC",  
12  "dependencies": {  
13    "express": "^5.1.0"  
14  },  
15  "devDependencies": {  
16    "nodemon": "^3.1.10"  
17  }  
18 }
```



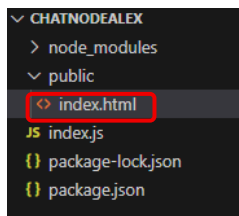


Ahora es momento de crear el archivo `index.js`, dentro de la carpeta principal (en mi caso `ChatNodeAlex`).

En el archivo `index.js`, agregar el siguiente código que levanta un servidor con Express en el puerto 4000 y sirve archivos estáticos desde la carpeta `public`.

```
JS index.js > ...
1  const express = require('express');
2  const app = express();
3
4  app.use(express.static('public'));
5
6  app.listen(4000, () =>{
7    console.log('Servidor corriendo en http://localhost:4000');
8  });
```

Bien, vamos a generar la vista, para ello, crear una carpeta llamada `public` dentro de la carpeta principal. Luego ahí, crear un archivo llamado `index.html` y agregar las siguientes etiquetas.



```
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4    <meta charset="UTF-8">
5    <title>Práctica de evaluación Socket</title>
6    <link rel="stylesheet" href="styles.css">
7  </head>
8  <body>
9
10   <div id="panel-bienvenida">
11     <h2>Práctica de evaluación usando Sockets de comunicación - Ing. Alex R.G.</h2>
12     <input id="persona" type="text" placeholder="Ingresa tu nombre">
13     <button id="ingresar" onclick="ingresarAlChat()">Ingresa al chat</button>
14   </div>
15
16   <div id="app-chat" style="display:none;">
17     <h2>Práctica de evaluación 1er. parcial - Ing. Alex R.G.</h2>
18     <h2 class="nube box">Chat con socket</h2>
19     <div id="ventana-mensajes">
20       <div id="output"></div>
21       <div id="escribiendo-mensaje"></div>
22     </div>
23     <input id="usuario" type="text" placeholder="Tu nombre">
24     <input id="mensaje" type="text" placeholder="Ingresa un mensaje">
25     <button id="enviar">Enviar</button>
26   </div>
27
28   <script src="/chat.js"></script>
29
30 </body>
31 </html>
```





Vamos a mejorar la vista con unos estilos. Para ello crear dentro de la carpeta de public, el archivo styles.css y agregar los siguientes atributos.

```
1 body{
2   font-family: 'Nunito';
3 }
4
5 h2{
6   font-size: 18px;
7   padding: 10px 20px;
8   color: #337ab7;
9   text-align: center;
10  font-family: Tahoma, Geneva, sans-serif;
11 }
12
13 #panel-bienvenida{
14   padding: 20% 20%;
15 }
16
17 #usuario{
18   color: #DE9631;
19   font-weight: bold;
20   font-size: 20px;
21   border: 1px solid #ddd;
22 }
23
24 #persona{
25   border: 1px solid #ddd;
26 }
27
28 #app-chat{
29   /*display: none;*/
30   max-width: 600px;
31   margin: 30px auto;
32   border: 1px solid #ddd;
33   box-shadow: 1px 3px 5px rgba(0,0,0,0.05);
34   border-radius: 2px;
35 }
```

```
36
37 #ventana-mensajes{
38   height: 400px;
39   overflow: auto;
40   background: #f9f9f9;
41   border: 1px solid #ddd;
42 }
43
44 #output p{
45   padding: 14px 0px;
46   margin: 0 20px;
47   border-bottom: 1px solid #e9e9e9;
48   color: #555;
49 }
50
51 /* Cuando usuario esta escribiendo mensaje */
52 #escribiendo-mensaje p{
53   color: #aaaaaa;
54   padding: 14px 0px;
55   margin: 0 20px;
56 }
57
58 /* Nombre de usuario */
59 #output strong{
60   color: #337ab7;
61 }
62
63 label{
64   box-sizing: border-box;
65   display: block;
66   padding: 10px 20px;
67 }
68
69 input{
70   padding: 10px 20px;
```





```

71  box-sizing: border-box;
72  background: #eee;
73  border: 0;
74  display: block;
75  width: 100%;
76  background: #fff;
77  border-bottom: 1px solid #eee;
78  font-family: Nunito;
79  font-size: 16px;
80  }
81  input:focus {outline:0;}
82
83  button{
84    background: #337ab7;
85    color: #ffffff;
86    font-size: 18px;
87    border: 0;
88    padding: 12px 0;
89    width: 100%;
90    cursor: pointer;
91    border: 1px solid transparent;
92    border-color: #2e6da4;
93    border-radius: 4px;
94  }
95
96  button:hover{
97    background: #246AA6;
98    color: #FAF7F7;
99  }
100
101  .nube{
102    background-color: #337ab7;
103    border-radius: 5px;
104    box-shadow: 0 0 6px #B2B2B2;
105    display: inline-block;
106    padding: 10px 18px;

```

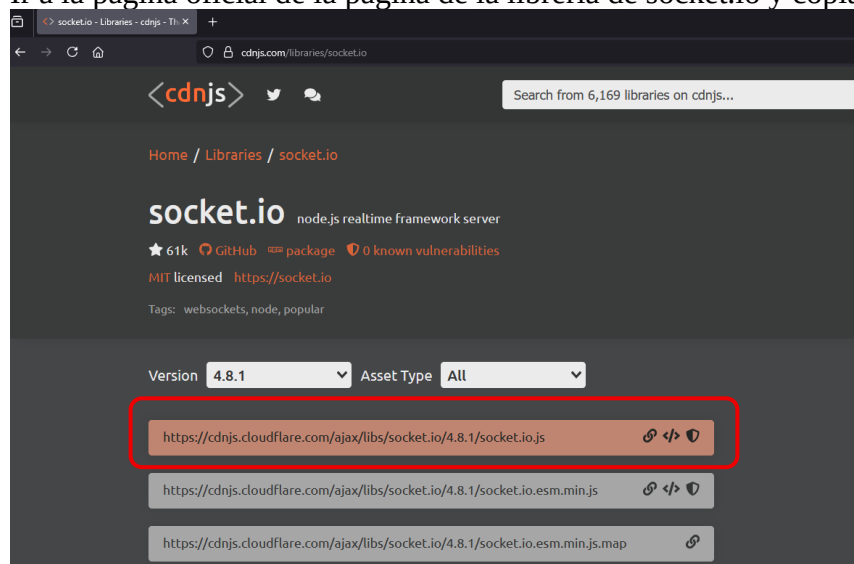
```

107  position: relative;
108  vertical-align: top;
109  color: #ffffff;
110  font-family: "Lucida Sans Unicode", "Lucida Grande", sans-serif;
111  }
112
113  .nube::before {
114    background-color: #337ab7;
115    content: "\00a0";
116    display: block;
117    height: 16px;
118    position: absolute;
119    top: 11px;
120    transform: rotate( 29deg ) skew( -35deg );
121    -moz-transform: rotate( 29deg ) skew( -35deg );
122    -ms-transform: rotate( 29deg ) skew( -35deg );
123    -o-transform: rotate( 29deg ) skew( -35deg );
124    -webkit-transform: rotate( 29deg ) skew( -35deg );
125    width: 20px;
126  }
127
128  .box {
129    float: right;
130    margin: 5px 45px 5px 20px;
131  }
132
133  .box::before {
134    box-shadow: -2px 2px 2px 0 #rgba( 178, 178, 178, .4 );
135    left: -9px;
136  }

```

Hasta ahora tenemos la interfaz gráfica de la aplicación. Es momento de establecer la conexión del lado del cliente con el servidor, utilizando socket.io.

Ir a la página oficial de la página de la librería de socket.io y copiar el link





Pegar ese link en el archivo index.html

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <title>Práctica de evaluación Socket</title>
6   <link rel="stylesheet" href="styles.css">
7   <script src="https://cdnjs.cloudflare.com/ajax/libs/socket.io/4.8.1/socket.io.js"></script>
8 </head>
9 <body>
```

Para la instalación de socket.io en el servidor, ir a la terminal y ejecutamos el siguiente comando

```
PS C:\ChatNodeAlex> npm install socket.io --save

added 25 packages, and audited 121 packages in 3s

20 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\ChatNodeAlex> 
```

Modificaremos el archivo index.js del servidor, para agregar la librería de socket.io que acabamos de descargar, el cual lo usa para comunicación en tiempo real. Ahora se maneja eventos de conexión, para envío de mensajes a todos los clientes y notificaciones de escritura al resto.





```
JS index.js > ...
1  var express = require('express');
2  var socket = require('socket.io');
3
4  var app = express();
5  var server = app.listen(4000, function(){
6    console.log('Servidor corriendo en http://localhost:5000');
7  });
8
9  app.use(express.static('public'));
10
11 var io = socket(server);
12
13 io.on('connection', function(socket){
14   console.log('Hay una conexion', socket.id);
15
16   socket.on('chat', function(data){
17     console.log(data);
18     io.sockets.emit('chat', data);
19   });
20
21   socket.on('typing', function(data){
22     socket.broadcast.emit('typing', data);
23   });
24 });
```

Bien, ahora se programará la funcionalidad, para ellos crear dentro de la carpeta public, el archivo llamado chat.js y agregar el siguiente código





```
1 var socket = io.connect('http://localhost:4000');
2 var persona = document.getElementById('persona'),
3     appChat = document.getElementById('app-chat'),
4     panelBienvenida = document.getElementById('panel-bienvenida'),
5     usuario = document.getElementById('usuario'),
6     mensaje = document.getElementById('mensaje'),
7     botonEnviar = document.getElementById('enviar'),
8     escribiendoMensaje = document.getElementById('escribiendo-mensaje'),
9     output = document.getElementById('output');
10
11 botonEnviar.addEventListener('click', function(){
12     if(mensaje.value){
13         socket.emit('chat', {
14             mensaje: mensaje.value,
15             usuario: usuario.value
16         });
17     }
18     mensaje.value = '';
19 });
20
21 mensaje.addEventListener('keyup', function(){
22     if(persona.value){
23         socket.emit('typing', {
24             nombre: usuario.value,
25             texto: mensaje.value
26         });
27     }
28 });
29
30 socket.on('chat', function(data){
31     escribiendoMensaje.innerHTML = '';
32     output.innerHTML += '<p><strong>' + data.usuario + ': </strong>' + data.mensaje + '</p>';
33 });
34
35 socket.on('typing', function(data){
36     if(data.texto){
37         escribiendoMensaje.innerHTML = '<p><em>' + data.nombre + ' esta escribiendo un mensaje...</em></p>';
38     }else{
39         escribiendoMensaje.innerHTML = '';
40     }
41 });
42
43 function ingresarAlChat(){
44     if(persona.value){
45         panelBienvenida.style.display = "none";
46         appChat.style.display = "block";
47         var nombreDeUsuario = persona.value;
48         usuario.value = nombreDeUsuario;
49         usuario.readOnly = true;
50     }
51 }
```

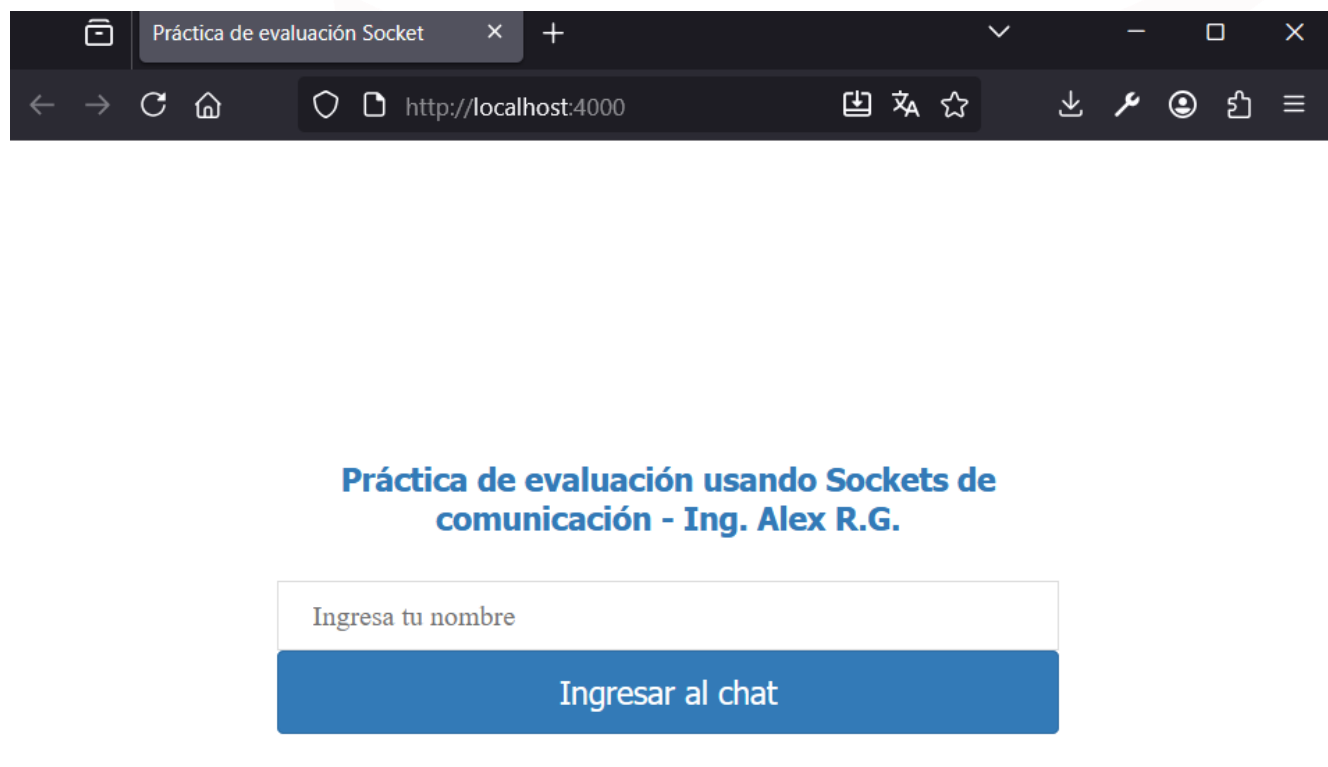
Una vez finalizado la codificación, se requiere iniciar el servidor con el comando npm start





```
PS C:\ChatNodeAlex> npm start  
  
> chatnodealex@1.0.0 start  
> node index.js  
  
Servidor corriendo en http://localhost:4000
```

Deben de tener la siguiente interfaz donde pide el usuario



Práctica de evaluación Socket

http://localhost:4000

Práctica de evaluación usando Sockets de comunicación - Ing. Alex R.G.

Ingresa tu nombre

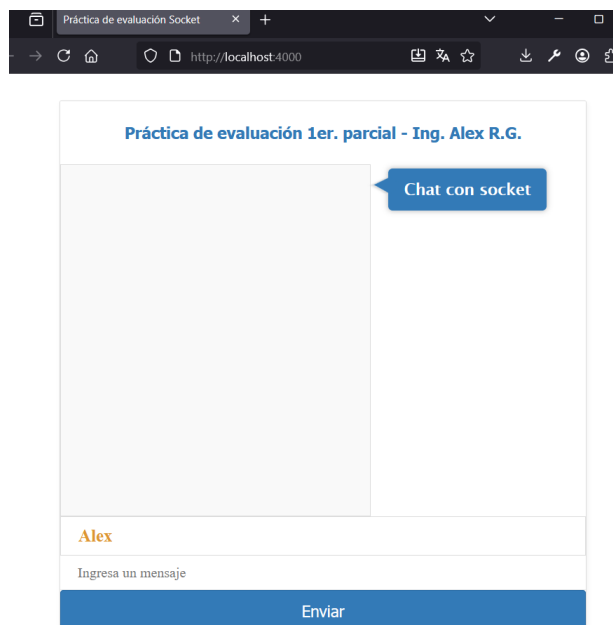
Ingresar al chat

Ingresa un nombre de usuario y da click en el botón Ingresar al chat

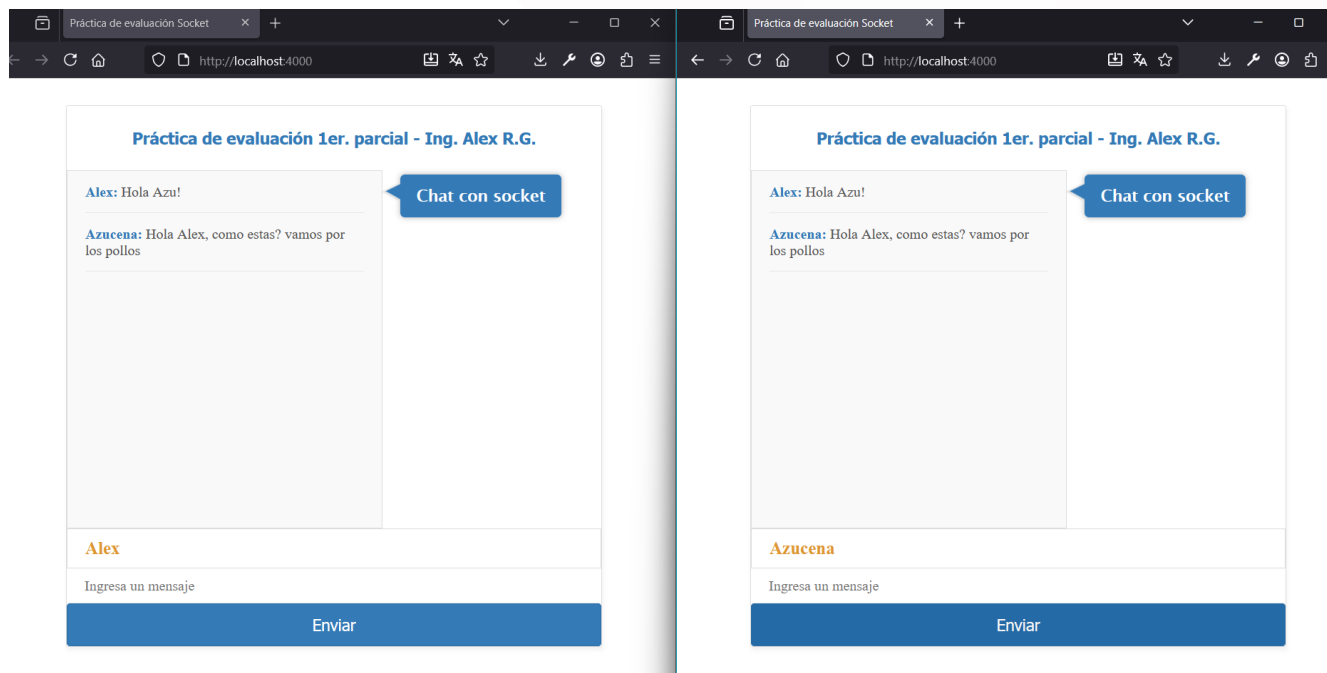




Mostrará la siguiente ventana



Abran varias pestañas o ventanas del navegador e ingresen diferentes nombres. Ahora podrán chatear entre los usuarios en un entorno web





Con esta práctica de evaluación del primer parcial, finalizamos el tema de Sockets de comunicación.

En clase te expliqué lo que quiero como complemento (subir este proyecto a su repositorio de Git y clonarlo en una instancia creada en AWS con Linux Ubuntu, para que este proyecto este alojado y funcionando en la nube con su respectiva dirección IP pública). Agregar sonido en cada envío de mensajes y modificar el código para que pueda adjuntar archivo (fotos, videos, documentos, etc)

Saludos.
Inge. Alex!

